

Due: Friday, Oct 16th at 3pm

The goal of this assignment is to give you practice with the basics of Python and to get you to think about how to translate a few simple algorithms into code. You will be allowed to work in pairs on some of the later assignments, but *you must work alone on this assignment*.

Epidemics and contagion are incredibly complex phenomena, involving both biological and social factors. Computer models, though imperfect, can offer insight into disease spread, and can represent infection with varying degrees of complexity.

SIR is a simple, but commonly used, model for epidemics. In the SIR model, a person can be in one of three states: *Susceptible* to the disease, *Infected* with the disease, or *Recovered* from the disease after infection (the model is named after these three states: S-I-R). In this model, we focus on a *network* of people, such as a community that could be experiencing an epidemic. Although simple, the SIR model captures that both social factors (like the shape of the network, e.g., how often people in the network interact with each other) and biological factors (like the duration of infection) that mediate disease spread.

In this assignment, you will write code to simulate a simplified version of the SIR epidemic model. Your code will model how infection spreads through a city from residents to their neighbors. At a high level, your code will iteratively calculate the disease states in a city day-by-day, keeping track of the state of each person until the end of the simulation. In addition, you will see how to use functions that build on one another to simplify a complex modeling process.

Getting started

Before you start working on the assignment's tasks, please take a moment to follow the steps described in [Coursework Basics](#) page to get the files for this assignment (these steps will be the same as the ones you followed to get the files for Short Assignment #1). Please note that you will not be able to start working on the assignment until you fetch the assignment files (which will appear in a `pa1` directory in your repository)

You should make sure that, as described on that page, you start an IPython session to experiment with your code. Again, the steps are similar to those for the short exercises: Open up a new terminal window and navigate to your `pa1` directory. Then, fire up `ipython3` from the **Linux command-line**, set up autoreload, and import your code as follows:

```
$ ipython3
```

```
In [1]: %load_ext autoreload
```

```
In [2]: %autoreload 2
```

```
In [3]: import sir
```

Finally, for this assignment, you may assume that the input passed to your functions has the correct format. *You may not alter any of the input that is passed to your functions*. In general, it is bad style to modify a data structure passed as input to a function, unless that is the explicit purpose of the function. If someone calls your function, they might have other uses for the data and should not be surprised by unexpected changes.

The Model

To begin building our SIR model, we must specify the model's details:

- **Disease states:** the ways of describing the the health of each person in the simulation.
- **Structure of a city:** How a city is represented, and the neighbors of each individual in the city.
- **Transmission rules** for disease spread within the city,
- **Contagion rules:** the rules for recovering and acquiring immunity to disease, and
- **Stopping conditions:** When to stop the simulation.

We specify each of these details below.

Disease states: all people in the simulation can exist in one of three states, *Susceptible*, *Infected*, or *Recovered*.

- *Susceptible*: the individual is healthy but may become infected in the future. We will use 'S' to represent susceptible individuals.
- *Infected*: the individual has an infection currently. We will represent these individuals with 'I0', 'I1', 'I2', etc. The number after the I represents how many days the individual has been infected (see "Contagion Rules," below).
- *Recovered*: the individual has recovered from an infection and will be immune to the infection for the rest of the simulation. We represent these individuals with 'R'. (Some versions of the SIR model remove recovered people from the model. In our model, recovered people will remain in the city.)

Please note that, in Tasks 6 and 7 of the assignment, we will introduce an additional state: *Vaccinated* (represented with 'V'). You do not need to worry about this state, or any references to vaccinations, until you get to those final tasks.

Structure of a city: a city in this simulation is represented as a list of people, each represented by a disease state. For example, a city of ['S', 'I1', 'R'] is composed of three people, the first of whom is susceptible, the second of whom is infected (and specifically, is one day into the infection), and the third of whom is recovered.

You can assume that every city has at least one person.

A person in our simplified model has up to two neighbors, the person immediately before them in the list (known as their *left* neighbor) and the person immediately after them in the list (known as their *right* neighbor). The first person in the list does not have a left neighbor and the last person in the list does not have a right neighbor. For example, consider the following list of people: ['Mark', 'Sarah', 'Lorraine', 'Marshall']:

- Mark has one neighbor: Sarah.
- Sarah has two neighbors: Mark and Lorraine.
- Lorraine has two neighbors: Sarah and Marshall
- Marshall has one neighbor: Lorraine.

Transmission rules: infection always spreads from infected people ('I0', 'I1', etc.) to susceptible people ('S'). In other words, a susceptible person with at least one infected neighbor will always get infected the next day.

Contagion rules: The number of days a person is infected and remains contagious is a parameter to the simulation. We will track the number of days a person has been infected as part of their state. People who become infected start out in state 'I0'. For each day a person is infected, we increment the counter by

one: 'I0' becomes 'I1', 'I1' becomes 'I2', etc. When the counter reaches the specified number of days contagious, we will declare them to be recovered ('R') and no longer contagious. At that point, they are immune to the disease and cannot become re-infected. For example, if we are simulating an infection in which people are contagious for three days, a newly infected person will start in state 'I0', move to 'I1' after one day, to 'I2' after two days, and to state 'R', where they will remain for the rest of the simulation, after three days.

Stopping conditions: the simulation should stop when there are no more infected people in the city.

Your tasks

For this assignment, we will specify a set of functions that you must implement. Like the first Short Exercises, understanding functions is not essential to completing this assignment, and we have specified exactly where in the file you need to add your code.

You will start with basic functions and work your way up to more complex tasks. We will also supply extensive test code. Over the course of the term, we will provide less and less guidance on the appropriate structure for your code.

Task 1: Count the number of infected people in a city

In Python, it is common to write helper functions that encapsulate key definitions and are only a few lines long. Your first task is to complete one such function: `count_infected`.

Here is the code you will see in the `sir.py` file:

```
def count_infected(city):
```

```
    """
```

```
        Count the number of infected people
```

```
    Inputs:
```

```
        city (list of strings): the state of all people in the  
        simulation at the start of the day
```

```
    Returns (int): count of the number of people who are  
        currently infected
```

```
    """
```

```
    # YOUR CODE HERE
```

```
    # REPLACE -1 WITH THE APPROPRIATE INTEGER
```

```
    return -1
```

The function docstring (between triple quotes) specifies the inputs to the function. You will be learning more about this as we cover functions in class but, for this assignment, it is enough to assume that `city` will contain the value

specified in the docstring and, more specifically, a list of strings with the state of all people in the simulation at the start of the day.

You must then write code that takes this city variable, and counts the number of infected neighbors. You must then replace the -1 in `return -1` with the appropriate value. For example, if your code uses a variable `num_infected` to count up the number of infected neighbors, you would replace `return -1` with `return num_infected`.

For example, given city `['I0', 'I0', 'I2', 'S', 'R']`, the function would return 3 (notice how we have to account for the fact that there are multiple infected states). Given a city such as `['S', 'S', 'S', 'S']`, the function would return 0.

Testing Task 1

Like the Short Exercises, we have provided a suite of automated tests for this assignment. You should take a moment to review the [Testing Your Code](#) page to understand how to run these tests, as well as the importance of doing some manual testing before you jump to the automated tests.

In particular, we suggest you start with some manual testing from IPython. Here, for example, are some sample calls to `count_infected`:

```
In [6]: sir.count_infected(['I0', 'I0', 'I2', 'S', 'R'])
```

```
Out[6]: 3
```

```
In [7]: sir.count_infected(['S', 'S', 'S', 'S'])
```

```
Out[7]: 0
```

If you get a `ModuleNotFoundError`, make sure you remembered to run `import sir` in IPython, so you can run the code contained in `sir.py`.

Once you're ready to run the automated tests, we have provided 15 test cases for you. The tested cities vary in size from one person to twenty people and have different mixes of disease states (e.g., all susceptible, all recovered, some infected with different number of days infected, etc).

Tests for `count_infected`

City	Expected result	Description
<code>['I0']</code>	1	One person city with an infected person.
<code>['I2000']</code>	1	One person city with an infected person who has a large days-infected count.
<code>['R']</code>	0	One person city with a recovered person
<code>['S']</code>	0	One person city with susceptible person

Tests for count_infected

City	Expected result	Description
['S', 'S', 'S', 'S']	0	Small city with all susceptible
['R', 'R', 'R', 'R']	0	Small city with all recovered
20 person city	0	Larger city with mix of susceptible and recovered
['I1', 'S', 'S', 'S']	1	Small city with one infected in slot 0, rest susceptible
['S', 'I1', 'S', 'S']	1	Small city with one infected in slot 1, rest susceptible
['S', 'S', 'I1', 'S']	1	Small city with one infected in slot 2, rest susceptible
['S', 'S', 'S', 'I1']	1	Small city with one infected in slot 3, rest susceptible
['I1', 'R', 'R', 'R']	1	Small city with one infected in slot 0, rest recovered
['I0', 'S', 'I1', 'R']	2	Small city with mixed types
20 person city	20	Larger city with all in state 'I0'
20 person city	20	Larger city with a mix of different infection states

You can also find the information from the table above in the file `count_infected.json` in the `tests/` subdirectory. For each function we ask you to write, we provide tests that are enumerated in a corresponding *json* file. You must not edit this file, nor do you need to understand the exact format of these files, but may need to occasionally refer to them (e.g., if you wanted to see the exact 20-person cities used in the last two test cases, you will find them in the `count_infected.json` file).

Our goal is to ensure sufficient *test coverage*, meaning that our tests account for as many different cases as possible in our code. For example, we could be tempted to write tests just for the following two cities:

- ['S', 'I0', 'I0', 'S', 'R']
- ['S', 'S', 'S', 'S']

However, what if we wrote a solution that forgot to account for infected states other than I0 or that assumed that the number of days infected would always be in the single digits? Neither of the above tests would *cover* such cases.

To run the tests for this task, simply run the following:

```
$ py.test -xvk count
```

Not sure what this command is doing, or what the output means? Make sure to review the [Testing Your Code](#) page for more details.

Debugging suggestions and hints for Task 1

Remember to save any changes you make to your code in your editor as you are debugging. Skipping this step is a common error. Fortunately, we've eliminated another common error – forgetting to reload code after it changes – by using the autoreload package. (If you skipped the [Getting started](#) section, please go back and follow the instructions to set up autoreload and import sir)

Task 2: Is a neighbor infected?

Next, you will write a function called `has_an_infected_neighbor` that will determine whether a susceptible person at a given position in a list has at least one neighbor who is infected.

More specifically, given the city and the person's position, your code will compute the positions of the specified person's left and right neighbors in the city, if they exist, and determine whether either one is in an infected state.

Recall that the first person in the city has a right neighbor, but not a left neighbor and the last person in the city has a left neighbor, but not a right neighbor. Your code will need to handle these special cases.

It only makes sense to call this function on a position that contains a susceptible person so, when you look at the code, you will see that we included the following line:

```
assert city[position] == "S"
```

to verify that the function has been called on a person who is susceptible to infection. In general, assertions have the following form:

```
assert <boolean expression>
```

Assertions are a useful way to check that your code is receiving valid inputs: if the boolean expression specified as the assertion's condition evaluates to False, the assertion statement will make the function fail. Simple assertions can greatly simplify the debugging process by highlighting cases where a function is being called incorrectly.

Testing for Task 2

As in the previous task, we suggest you start by trying out your code in `ipython3` before you run the automated tests. Here, for example, are some sample calls to `has_an_infected_neighbor`:

```
In [8]: sir.has_an_infected_neighbor(['I1', 'S', 'S'], 1)
```

```
Out[8]: True
```

```
In [9]: sir.has_an_infected_neighbor(['S', 'I1', 'IO'], 0)
```

```
Out[9]: True
```

```
In [9]: sir.has_an_infected_neighbor(['S', 'R', 'IO'], 0)
```

Out[9]: False

In [10]: sir.has_an_infected_neighbor(['S', 'I0', 'S'], 2)

Out[10]: True

In [10]: sir.has_an_infected_neighbor(['S'], 0)

Out[10]: False

In the first sample call, we are checking whether the susceptible person in position 1 has an infected neighbor. Since their left neighbor (at position 0) is infected, the result should be True.

The next call checks whether the susceptible person in position 0 has an infected neighbor. This person does not have a left neighbor. Their right neighbor, at position 1, though, is infected and so, the result should be True.

The third call also checks the person at position 0. In this case, the person at position 1 is not infected, and so the expected result is False.

The fourth call checks the person at position 2. This person does not have a right neighbor. Their left neighbor, at position 1, is infected, though, and so, the expected result is True.

Finally, the last call will return False. Why? Because, the lone person in this city has no neighbors and so, by definition has no infected neighbors. Take into account that a correct solution does not need include a condition that checks “if this city has just one person”. The code should work for cities of all sizes, including cities with a single person. Hint: the person in this one-person city is both the *first* and *last* element of the list.

The table below provides information about the tests for `has_an_infected_neighbor`. Each row contains the values that will be passed for the city and position arguments for that test, the expected result, and a brief description of the tests purpose. You can also find this data in `tests/has_infected_neighbor_tests.json`.

Tests for `has_an_infected_neighbor`

City	Position	Expected result	Description
['I0', 'S', 'S']	1	True	Left neighbor infected.
['I1000', 'S', 'S']	1	True	Left neighbor infected w/ multi-digit days infected.
['R', 'S', 'I0']	1	True	Right neighbor infected.
['R', 'S', 'I1000']	1	True	Right neighbor infected w/ multi-digit days infected.
['I1', 'S', 'I0']	1	True	Both neighbors infected

Tests for has_an_infected_neighbor

City	Position	Expected result	Description
['S', 'S', 'R']	1	False	Neither neighbor infected.
['R', 'S', 'S', 'I1']	2	True	City with more than three people. Right neighbor infected.
['R', 'I200', 'S', 'R']	2	True	City with more than three people. Left neighbor infected.
['I0', 'S', 'S', 'R']	2	False	City with more than three people. Neither neighbor infected.
['S', 'S', 'S', 'I1']	0	False	First person, Single neighbor (right) not infected.
['S', 'I1', 'S', 'I1']	0	True	First person, Single neighbor (right) infected.
['I0', 'S', 'S', 'S']	3	False	Last person, Single neighbor (left) not infected
['I0', 'S', 'I10', 'S']	3	True	Last person, Single neighbor (left) infected
['S']	0	False	Solo person in city.

You can run these tests by running the following command from the **Linux command-line**:

```
$ py.test -xvk has
```

Debugging suggestions and hints for Task 2

There is a lot going on in this function and, when you are debugging, it can be helpful to know exactly what is happening inside the function. print statements are among the most intuitive ways to identify what your code is actually doing and will become your go-to debugging method. If you are struggling to get started or to return the correct values from your function, consider the following debugging suggestions:

- Print which neighbors exist;
- Print the positions you calculated for those neighbors; and
- Print the values you extracted for those neighbors.

Is your code behaving as expected given these values?

Also, make sure that you are returning, not printing, the desired value from your function.

Don't forget to remove your debugging code (i.e., the print statements) before you submit your solution.

Task 3: Advance person at position

Your third task is to complete the function `advance_person_at_position`. The goal of this function is to advance the disease state of a person from one day to the next. Given a city, a person's location within that city, and the number of days c the infection is contagious, your function should determine the next state for the person. Specifically, if the person is:

1. Susceptible ('S'): you need to determine whether they have an infected neighbor (by using the `has_an_infected_neighbor` function) and, if so, change them to the first infected state ('I0'). Otherwise, they remain in the Susceptible ('S') state.
2. Infected ('I', followed by an integer; we will refer to that integer as x): determine whether the person remains infected (that is, $x < c$) and moves to the next infected state (e.g. 'I0' becomes 'I1', 'I1' becomes 'I2', etc) or switches to the recovered state ('R'). To compute the new state of an infected person, you will need to extract the number of days infected from the state as a string, convert it to an integer, and then compare it to the number of days contagious c . If you determined the person will remain infected, you'll need to construct a new string from 'I' and $x+1$.
3. Recovered ('R'): you should do nothing. Recovered people remain in that state.

As an example, consider the following calls to `advance_person_at_position`:

```
In [22]: sir.advance_person_at_position(['I0', 'I1', 'R'], 0, 2)
```

```
Out[22]: "I1"
```

```
In [22]: sir.advance_person_at_position(['I0', 'I1', 'R'], 1, 2)
```

```
Out[22]: "R"
```

```
In [22]: sir.advance_person_at_position(['I0', 'I1', 'R'], 2, 2)
```

```
Out[22]: "R"
```

The first call determines that the person at position 0 moves from state 'I0' to 'I1'. The second call determines that the person at position 1 shifts to state 'R', because the parameters specify that a person is only contagious for two days. And finally, the third call returns 'R' because the person at position 2 is already in state 'R'.

Testing for Task 3

The table below provides information about the tests for `advance_person_at_position`. Each row contains the values that will be passed for the city, position, and days_contagious arguments for that test, the expected result, and a brief description of the test.

Tests for `advance_person_at_position`

City	Position	Days Contagious	Result	Description
['I1', 'S', 'S']	1	3	I0	Left neighbor is infected, susceptible person gets infected.

Tests for advance_person_at_position

City	Position	Days Contagious	Result	Description
['S', 'S', 'I0']	1	3	I0	Right neighbor is infected, susceptible person gets infected.
['I20', 'S', 'I0']	1	3	I0	Both neighbors are infected, susceptible person gets infected.
['R', 'S', 'R']	1	3	S	Neither neighbor is infected, susceptible person does not get infected.
['I1', 'S', 'S', 'S']	2	3	S	Neither neighbor is infected, susceptible person does not get infected.
['S', 'S', 'I0']	0	3	S	Right neighbor only, susceptible person does not get infected.
['S', 'I1500', 'I0']	0	3	I0	Right neighbor only, susceptible person gets infected.
['I1', 'R', 'S']	2	3	S	Left neighbor only, susceptible person does not get infected.
['I1', 'I1500', 'S']	2	3	I0	Left neighbor only, susceptible person gets infected.
['I1', 'I1500', 'S']	0	3	I2	Infected should be incremented.
['I2', 'I1500', 'S']	0	3	R	Infected should be converted to recovered.
['I2', 'I1500', 'S']	1	2000	I1501	Infected should be incremented. Large number of days contagious.
['I2', 'I1500', 'S']	1	1501	R	Infected person recovers. Large number of days contagious.
['I2', 'I1500', 'R']	2	2000	R	Recovered, no change.

You can run these tests by executing the following command from the Linux command-line:

```
$ py.test -xvk advance
```

Task 4: Move the simulation forward a single day

Your fourth task is to complete the function `simulate_one_day`. This function will model one day in a simulation and will act as a helper function to `run_simulation`. More concretely, `simulate_one_day` should take the city's state at the start of the day and the number of days a person is contagious `c` and return a new list of disease states (i.e., the state of the city after one day).

Your implementation for this function must use `advance_person_at_position` to determine the new state of each person in the city.

For example:

In [24]: `sir.simulate_one_day(['S', 'I0', 'S'], 2)`

Out[24]: `['I0', 'I1', 'I0']`

Notice how the susceptible people at positions 0 and 2 both become infected (they both have an infected neighbor) and the person at position 1 advances to the next state of their infection ('I0' to 'I1').

Testing for Task 4

The table below provides information about the tests for `simulate_one_day`. Each row contains the values that will be passed for the `city` and `days_contagious` arguments for that test, the expected result, and a brief description of the test.

Tests for `simulate_one_day`

City	Days contagious	Expected result	Description
['I0', 'I1', 'I100']	200	['I1', 'I2', 'I101']	Are the I values are incremented correctly?
['I2', 'I2', 'I2']	3	['R', 'R', 'R']	Are the I values are converted to R correctly?
['R', 'R', 'R']	3	['R', 'R', 'R']	R values should not change.
['I1', 'S', 'I1']	3	['I2', 'I0', 'I2']	Susceptible person becomes infected
['I1', 'S', 'I1']	2	['R', 'I0', 'R']	A susceptible person becomes infected, even when its neighbors recover in the same day.
['S', 'I0', 'S']	2	['I0', 'I1', 'I0']	Two susceptible people become infected.
['S', 'S', 'S']	2	['S', 'S', 'S']	None of the susceptible people become infected
30 person city	2	See test	Large city w/ medium infection rate

You can run these tests by executing the following command from the **Linux command-line**:

```
$ py.test -xvk one
```

Debugging suggestions for Task 4

If you are struggling to get started or to return the correct values in your function, consider printing out each person's old and new disease states and ensuring that the new disease states are correct in all cases.

Task 5: Run the simulation

Your fifth task is to complete the function `run_simulation`, which takes the starting state of the city and the number of days a person is contagious, and returns both the final state of the city and the number days simulated as a tuple. You will notice this function also takes two additional optional parameters (`random_seed` and `vaccine_effectiveness`); you can ignore these parameters as we will not use them until the next task.

The function must run one whole simulation, repeatedly calling `simulate_one_day` until you reach the stopping condition of the simulation: when the city has no infected people in it. As you do this, the function must also count the number of days simulated.

Take into account that, if the stopping condition is true at the start of the simulation, then the number of days simulated will be zero.

Here are two example uses of this function:

In [32]: `sir.run_simulation(['S', 'S', 'I0'], 3)`

Out[32]: `(['R', 'R', 'R'], 5)`

In [33]: `sir.run_simulation(['S', 'R', 'I0'], 3)`

Out[33]: `(['S', 'R', 'R'], 3)`

Testing Task 5

We have provided five tests for this task.

Tests for `run_simulation`

Starting City	Days contagious	Expected Result: city, number of days simulated	Description
['S', 'S', 'I0']	3	(['R', 'R', 'R'], 5)	Everyone is infected and recovers.
['S', 'R', 'I0']	3	(['S', 'R', 'R'], 3)	Only one infection, recovered person prevents susceptible person from getting infected.
['R', 'S', 'S']	2	(['R', 'S', 'S'], 0)	No infections in starting city, so we don't simulate any days.
['R', 'I0', 'S', 'I1', 'S', 'R', 'S']	10	(['R', 'R', 'R', 'R', 'R', 'R', 'S'], 11)	Medium city.

Tests for run_simulation

Starting City	Days contagious	Expected Result: city, number of days simulated	Description
30 person city	2	(See run_simulation_tests.json for expected city, 8)	Large city.

You can run these tests by executing the following command from the **Linux command-line**.

```
$ py.test -xvk run
```

Debugging hints for Task 5

If you are generating the wrong final state for the city, try printing the day (0, 1, 2, etc.), the disease states *before* the call to simulate_one_day, and the disease states *after* the call to simulate_one_day.

Task 6: Vaccinating a City

Your next task is to add support for a new state: Vaccinated ('V'). This will involve implementing the vaccinate_city function and updating the run_simulation function.

The 'V' state actually behaves exactly like the 'R' state: a vaccinated person is immune to the disease and cannot become infected. However, while the 'R' state is reached during the simulation after a person goes through an infection, the 'V' state is reached before the simulation starts, when we will vaccinate every susceptible person in the city.

However, no vaccine is 100% effective, which means that giving a susceptible person a vaccine does not change them unconditionally to the 'V' state. Instead, our simulation will have an additional parameter: a vaccine effectiveness rate v between 0.0 and 1.0.

In the context of this exercise, you can think about vaccine effectiveness as being similar to flipping a weighted coin. This means that, if v is 0.8, the coin will land on “heads” 80% of the time, and on “tails” 20% of the time. Now, imagine that the coin has “the vaccine confers immunity” instead of “heads”, and “the vaccine does NOT confer immunity” instead of “tails”.

So, for each susceptible person that receives a vaccine, we flip this weighted coin to determine whether the vaccine works (the person switches to the 'V' state) or does not work (the person remains in the 'S' state).

To “flip a coin” in Python, we will use a *random number generator* and, more specifically, you will call random.random(), a function that returns a random floating point number between 0.0 and 1.0. We will interpret the returned value as follows:

- If the random number is *strictly* less than v , the vaccine works.
- If the random number is greater than or equal to v , the vaccine does not work.

So, vaccinate_city will take a city and a vaccine effectiveness rate, and will return a new city where each susceptible person has been vaccinated according to the above rule. Once you’ve implemented vaccinate_city, you will need to modify run_simulation to call vaccinate_city *once* before simulating any days. You’ll notice that all our previous tests called run_simulation with vaccine_effectiveness set to 0.0, which means that, once you complete

this task, previous tests won't break because they will continue to behave as before: with no one in the city being vaccinated.

You will also have to make sure that `advance_person_at_position` works correctly with the 'V' state. In particular, someone in the 'V' state must remain in that state. You can test whether your function is working as expected like this:

```
In [3]: sir.advance_person_at_position(['S', 'V', 'S'], 1, 2)
```

```
Out[3]: 'V'
```

If the above call returns anything other than 'V', make sure you are correctly handling the 'V' state in `advance_person_at_position`.

Now, there's a small twist to using random numbers. Let's give `random.random()` a try; to do so, you will first have to import the random module:

```
In [1]: import random
```

Now, try calling the function a few times:

```
In [2]: random.random()
```

```
Out[2]: 0.595299247755262
```

```
In [3]: random.random()
```

```
Out[3]: 0.8159606343474648
```

```
In [4]: random.random()
```

```
Out[4]: 0.30061626031208444
```

Here's the tricky thing about random numbers: you will almost certainly see different numbers when you try out `random.random()` (which makes sense: the function is meant to return a random number!) This can complicate debugging and testing, because you can call a function that relies on `random.random()` (like `vaccinate_city`) and get different results every time.

Fortunately, we can ensure that `random.random()` returns the same sequence of numbers when it is called by initializing it with a *seed* value. It is common to set the seed value for a random number generator when debugging. If we do not actively set the seed, random number generators will usually derive one from the system clock.

Since many of our tests use the same seed (20170217), we have defined a constant, `TEST_SEED`, with this value in `sir.py` for your convenience. This value should be used for testing only; it should not appear anywhere in the code you write.

Let's try out setting the seed using the value of `sir.TEST_SEED` and then making some calls to the random number generator in `ipython3`:

```
In [11]: sir.TEST_SEED
```

```
Out[11]: 20170217
```

```
In [12]: random.seed(sir.TEST_SEED)
```

```
In [13]: random.random()
```

```
Out[13]: 0.48971492504609215
```

```
In [14]: random.random()
```

```
Out[14]: 0.23010566619210782
```

```
In [15]: random.seed(sir.TEST_SEED)
```

```
In [16]: random.random()
```

```
Out[16]: 0.48971492504609215
```

```
In [17]: random.random()
```

```
Out[17]: 0.23010566619210782
```

Notice that the third and fourth calls to `random.random()` generate exactly the same values as the first two calls. Why? Because we set the seed to the exact same value before the first and third calls.

This behavior of `random` has another implication: it is *crucial* that `vaccinate_city` call `random.random()` only when you encounter a susceptible person. If you call the random number generator for every person (including people not in the 'S' state), your code may generate different answers than ours on subsequent tasks.

Testing for Task 6

Unlike previous tasks, you have to be careful to initialize the random seed before calling `vaccinate_city`, to make sure you get the expected results. For example:

```
In [22]: random.seed(sir.TEST_SEED)
```

```
In [23]: sir.vaccinate_city(['S', 'S', 'S', 'S', 'S', 'I0', 'S'], 0.8)
```

```
Out[23]: ['V', 'V', 'V', 'V', 'S', 'I0', 'V']
```

```
In [24]: random.seed(sir.TEST_SEED)
```

```
In [25]: sir.vaccinate_city(['S', 'S', 'S', 'S', 'S', 'I0', 'S'], 0.3)
```

Out[25]: ['S', 'V', 'S', 'S', 'S', 'I0', 'S']

However, when testing your updated `run_simulation`, take into account that the function takes the random seed as a parameter, which means you need to call `random.seed` *inside* `run_simulation`. Here are some example uses:

In [34]: `sir.run_simulation(['S', 'S', 'S', 'S', 'S', 'I0', 'S'], 2, sir.TEST_SEED, 0.0)`

Out[34]: (['R', 'R', 'R', 'R', 'R', 'R', 'R'], 7)

In [35]: `sir.run_simulation(['S', 'S', 'S', 'S', 'S', 'I0', 'S'], 2, sir.TEST_SEED, 0.3)`

Out[35]: (['S', 'V', 'R', 'R', 'R', 'R', 'R'], 5)

In [36]: `sir.run_simulation(['S', 'S', 'S', 'S', 'S', 'I0', 'S'], 2, sir.TEST_SEED, 0.8)`

Out[36]: (['V', 'V', 'V', 'V', 'R', 'R', 'V'], 3)

Notice how these results make sense: as the effectiveness of the vaccine increases, the duration of the epidemic decreases.

The table below provides information about the automated tests for `vaccinate_city`. Each row contains the seed used to initialize the random number generator, the values that will be passed for the `city` and `vaccine_effectiveness` arguments for that test, and the expected result. The last column briefly describes the test.

Tests for `vaccinate_city`

Seed	City	Vaccine effectiveness	Expected result	Description
20170217	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	0.0	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	Completely ineffective vaccine. No one should be vaccinated.
20170217	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	1.0	['V', 'V', 'V', 'V', 'V', 'I0', 'V']	Completely effective vaccine. Every susceptible person should be vaccinated.
20170217	['I0', 'I1', 'I2', 'R']	1.0	['I0', 'I1', 'I2', 'R']	Completely effective vaccine, but no susceptible people. Everyone should stay in their original state.
20170217	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	0.3	['S', 'V', 'S', 'S', 'S', 'I0', 'S']	Partially effective vaccine. Only one person ends up vaccinated.
20170217	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	0.8	['V', 'V', 'V', 'V', 'S', 'I0', 'V']	Partially effective vaccine. All but one susceptible person ends up vaccinated.

Tests for vaccinate_city

Seed	City	Vaccine effectiveness	Expected result	Description
20170218	['S', 'S', 'S', 'S', 'S', '10', 'S']	0.8	['V', 'V', 'V', 'V', 'V', '10', 'V']	Partially effective vaccine, but with a different seed than Task 5, the outcome is different.

And this table provides information about a series of additional tests for run_simulation that (unlike the tests for Task 5) use a vaccination effectiveness rate greater than zero.

Tests for run_simulation (with vaccination)

Seed	Starting City	Days contagious	Vaccine effectiveness	Expected Result: city, number of days simulated	Description
20170217	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	0.0	[['R', 'R', 'R', 'R', 'R', 'R', 'R'], 7]	Completely ineffective vaccine (no one is vaccinated, like Task 5)
20170217	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	0.3	[['S', 'V', 'R', 'R', 'R', 'R', 'R'], 5]	Vaccine effectiveness (different seed)
20170218	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	0.3	[['S', 'S', 'S', 'V', 'V', 'R', 'R'], 3]	Vaccine effectiveness (different seed)
20170217	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	0.8	[['V', 'V', 'V', 'V', 'R', 'R', 'V'], 3]	Vaccine effectiveness (different seed)
20170218	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	0.8	[['V', 'V', 'V', 'V', 'V', 'R', 'V'], 2]	Vaccine effectiveness (different seed)
20170217	['S', 'S', 'S', 'S', 'S', '10', 'S']	2	1.0	[['V', 'V', 'V', 'V', 'V', 'R', 'V'], 2]	Completely effective vaccine

Tests for run_simulation (with vaccination)

Seed	Starting City	Days contagious	Vaccine effectiveness	Expected Result: city, number of days simulated	Description
20170217	30-person city	2	0.5	(See simulation_with_vaccine.json for expected city, 5)	Large city

You can run all these tests by executing the following command from the **Linux command-line**:

```
$ py.test -xvk vac
```

If you want to run *only* the tests for vaccinate_city or for the updated run_simulation, run one of the following:

```
$ py.test -xvk vaccinate_city
```

```
$ py.test -xvk simulation_with_vaccine
```

Debugging suggestions and hints for Task 6

If you are struggling to get started or to return the correct values in your function, consider the following suggestions to debug your code:

- Print the value returned by random.random().
- Make sure that you are making the right number of calls to random.random (you should only call it when you encounter an 'S' in the city).
- When testing in ipython3, ensure that you have reset the seed for the random number generator before each test call to vaccinate_city.
- But make sure you don't call random.seed(sir.TEST_SEED) anywhere in your code. In fact, you should only be calling random.seed inside run_simulation, and always with the provided random_seed parameter.

Task 7: Determining average time to zero infections

Your last task is to complete the function calc_avg_days_to_zero_infections, which computes the average number of days it takes for a city to reach zero infections. This function takes the starting state of the city, the number of days contagious, the random seed, the vaccine effectiveness rate, and the number of trials to run as arguments and returns the average number of days until a city reaches zero infections over the num_trials different trial runs. The number of days until a city reaches zero infections is simply the number of days returned by run_simulation.

Each time you run a trial simulation, you should increase the random seed by 1. *It is important that you increment your random seed.* If you forget to increment your seed, all trials will be identical, and if you increment your seed in a different way than specified, your code may produce a different result (and thereby, not pass our tests).

Your implementation should call run_simulation, which sets the seed, so unlike the previous task, you do not need to call random.seed before running this function in ipython3.

Here's a sample use of this function:

```
In [52]: sir.calc_avg_days_to_zero_infections(['S', 'S', 'S', 'S', 'S', 'I0', 'S'],
```

...: 2, sir.TEST_SEED, 0.65, 5)

Out[52]: 2.6

How did the function arrive at an average of 2.6 days? Here's a table that shows, for each trial, the seed used, the starting state, the end state, and the number of days until the city reaches zero infections.

Intermediate values from calc_avg_days_to_zero_infections

Simulation number	Seed	Starting state for simulation run	Final state for simulation run	Days to zero infections
0	20170217	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	['V', 'V', 'S', 'V', 'R', 'R', 'R']	3
1	20170218	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	['V', 'V', 'V', 'V', 'V', 'R', 'V']	2
2	20170219	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	['S', 'V', 'V', 'R', 'R', 'R', 'V']	4
3	20170220	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	['V', 'V', 'V', 'S', 'V', 'R', 'V']	2
4	20170221	['S', 'S', 'S', 'S', 'S', 'I0', 'S']	['V', 'V', 'S', 'S', 'V', 'R', 'V']	2

Because we change the random seed in each trial, we do not necessarily get the same outcome in each trial. Since the number of days across the trials is 3, 2, 4, 2, and 2, the average works out to be .

Testing Task 7

We have provided ten tests for this task. The first three can be checked easily with print statements. The fourth and fifth tasks use a large number of trials (100) and different seeds. You will see that as the number of trials increases, the starting seed matters less. The sixth and seventh tests use larger cities. And the last three tests check edge cases: one trial, a 100% effective vaccine, and a city without any susceptible people.

Tests for calc_avg_num_newly_infected

Starting Seed	Starting City	Days contagious	Vaccine effectiveness	Number of Trials	Expected result	Description
20170217	['S', 'I1', 'S', 'I0']	2	0.8	5	2.2	Test case that can be hand-com
20170217	['S', 'I1', 'S', 'I0']	2	0.3	5	2.8	A less effective vaccine results in longer epidemic
20170219	['S', 'I1', 'S', 'I0']	2	0.8	5	2.4	Different seed

Tests for calc_avg_num_newly_infected

Starting Seed	Starting City	Days contagious	Vaccine effectiveness	Number of Trials	Expected result	Description
20170217	['S', 'I1', 'S', 'I0']	2	0.8	100	2.31	Large number of trials.
20170218	['S', 'I1', 'S', 'I0']	2	0.8	100	2.31	Large number of trials with a different seed.
20170217	30 person city	2	0.8	10	3.5	30 person city, effective vaccine, few days contagious
20170217	49 person city	2	0.3	100	5.48	49 person city, less effective vaccine, few days contagious.
20170217	['S', 'S', 'I1', 'I1', 'I1', 'I1', 'S']	2	0.5	1	3.0	Edge case: 1 trial
20170217	['S', 'S', 'I1', 'I1', 'I1', 'I1', 'S']	2	1.0	10	1.0	Edge case: 100% effective vaccine
20170217	['R', 'R', 'R', 'R']	2	0.5	10	0.0	Edge case: population is already recovered, so all simulations should return zero days.

You can run these tests by executing the following command from the **Linux command-line**.

```
$ py.test -xvk avg
```

Putting it all together

We have included code in `sir.py` that calls your functions to run a single simulation or to calculate the average number of days for a city to reach zero infections.

Running this program with the `--help` flag shows the flags to use for different arguments.

```
$ python3 sir.py --help
```

Usage: `sir.py [OPTIONS] CITY`

Process the command-line arguments and **do** the work.

Options:

--days-contagious INTEGER
--random_seed INTEGER
--vaccine-effectiveness FLOAT
--num-trials INTEGER
--task-type [single|average]
--debug
--help Show this message and exit.

Cities are specified as a comma separated string, such as, "S, S, I0".

Here is a sample use of this program that runs a single simulation:

```
$ python3 sir.py "S, S, I0" --random_seed=20170217 --vaccine-effectiveness=0.5 --days-contagious=3 --task-type=single
```

and here is the output that it should print:

Running one simulation...

Final city: ['V', 'V', 'R']

Days simulated: **3**

Here is a sample use of this program that calculates the average number of days for the city to reach zero infections:

```
$ python3 sir.py "S, S, I0" --random_seed=20170217 --vaccine-effectiveness=0.5 --days-contagious=3 --num-trials=5 --task-type=average
```

Running multiple trials...

Over **5** trial(s), on average, it took **3.4** days **for** the number of infections to reach zero

Grading

Programming assignments will be graded according to a general rubric. Specifically, we will assign points for completeness, correctness, design, and style. (For more details on the categories, see our [PA Rubric page](#).)

The exact weights for each category will vary from one assignment to another. For this assignment, the weights will be:

- **Completeness:** 75%
- **Correctness:** 15%
- **Design:** 0%
- **Style:** 10%

The completeness part of your score will be determined using automated tests. To get your score for the automated tests, simply run the grader script, as described in our [Testing Your Code](#) page.

Cleaning up

Before you submit your final solution, you should, remove

- any print statements that you added for debugging purposes and
- all in-line comments of the form: “YOUR CODE HERE” and “REPLACE ...”

Also, check your code against the style guide. Did you use good variable names? Do you have any lines that are too long, etc.

Do **not** remove header comments, that is, the triple-quote strings that describe the purpose, inputs, and return values of each function.

As you clean up, you should periodically save your file and run your code through the tests to make sure that you have not broken it in the process.

Submission

You must submit your work through Gradescope (linked from our Canvas site). In the “Programming Assignment #1” assignment, simply upload file `sir.py` (do not upload any other file!). Please note:

- You are allowed to make as many submissions as you want before the deadline.
- Please make sure you have read and understood our [Late Submission Policy](#)
- Your completeness score is determined solely based on the automated tests, but we may adjust your score if you attempt to pass tests by rote (e.g., by writing code that hard-codes the expected output for each possible test input).
- Gradescope will report the test score it obtains when running your code. If there is a discrepancy between the score you get when running our grader script, and the score reported by Gradescope, please let us know so we can take a look at it.

Acknowledgments: This assignment was inspired by a discussion of the SIR model in the book [Networks, Crowds, and Markets](#) by Easley and Kleinberg. Emma Nechamkin wrote the original version of this assignment.